

# Optimizing In-network Caching for Key-Value Stores under Write-intensive Workloads

Yuanhang Gao, Yingwen Chen, Xiangrui Yang\*, Huan Zhou, Shihua Tang, Yuanfeng Chen, Ming Xu

*College of Computer Science and Technology*

*National University of Defense Technology, Changsha, China*

{gaoyh, ywch, yangxiangrui11, huanzhou, chenyanfeng, xuming}@nudt.edu.cn, tangyaoitangdou@126.com

**Abstract**—Key-value stores are critical for modern data centers, yet they struggle with performance degradation under write-intensive workloads due to frequent cache invalidations. Traditional read-optimized caching mechanisms fail to maintain efficiency as frequent updates lead to obsolete cached data. As a result, queries sent by clients will suffer from long queuing delays or even be dropped and overall system performance will be degraded severely. This paper presents a novel key-value stores architecture, named Freq-absorb, which addresses this issue through a delayed-write mechanism delaying writes commitment to backend servers. By buffering write queries within the switch for a specific period, Freq-absorb reduces the impact of cache invalidations, improves switch hit rates, and enhances overall system performance. We implement a prototype using an FPGA-based switch that acts as the ToR (Top of Rack) switch to achieve cache and the appended write buffer. Our experimental results show that compared to the read cache mechanism, our approach can reduce switch miss from 99% to 48.4% when handling write-intensive workloads, significantly enhancing performance.

**Index Terms**—Key-value stores, In-network processing, Distributed system

## I. INTRODUCTION

Key-value stores are essential components in modern data center architectures, providing the performance needed to manage vast amounts of data. These systems support a variety of critical services, from web search [1] to large-scale machine learning [2], making their efficiency vital to overall data center operations. Historically, key-value stores have been optimized for read-heavy workloads, with caching strategies primarily designed to enhance read performance by storing frequently accessed data in fast, accessible memory, even in the ToR (Top of Rack) switch. However, the increasing prevalence of write-intensive workloads (write ratio  $> 10\%$ ) [3][4] presents significant challenges, causing traditional read-optimized caches to become ineffective [5].

Write-intensive workloads generate frequent and substantial data updates, which disrupt the cache contents and lead to widespread cache invalidation. This invalidation undermines the effectiveness of read caches, as cached data becomes obsolete more rapidly than it can be refreshed. Consequently, the benefits of caching—reduced read latency and increased throughput—are significantly diminished, leading to degraded overall system performance. The core problem is

that traditional caching mechanisms, which excel in environments with read-heavy access patterns, struggle to maintain efficiency under write-heavy conditions. The high rate of cache invalidation forces systems to frequently fetch data from backend storage, negating the speed advantages of the in-network cache and resulting in increased read latencies and higher backend loads. This situation is exacerbated by the centralized nature of many network controller mechanisms, which become bottlenecks as they try to manage the synchronization and consistency of rapidly changing data.

To address these challenges, we propose an innovative solution within the network, named Freq-absorb architecture, specifically designed to mitigate the adverse effects of write-intensive workloads on read caches. Our approach centers on a delayed-write mechanism that leverages in-network processing capabilities of programmable hardware. This mechanism temporarily buffers frequent write queries within the switch data plane, reducing the immediate impact on cache contents and backend storage.

By buffering write queries in the data plane of the ToR switch and delaying their commitment to the backend servers, our solution effectively minimizes the rate of switch miss. This not only preserves the integrity and usefulness of cached data but also enhances overall system throughput and load reduction. Ensuring data integrity and reliability is a critical aspect of our solution. In scenarios where write queries are delayed, there is a risk of data loss or inconsistency, particularly during switch failures or state transitions. To mitigate these risks, we incorporate a lightweight reliability mechanism. This mechanism ensures that data remains consistent and reliable, even in the face of operational disruptions, maintaining the robustness of the system.

This paper provides a pertinent analysis of the workload characteristics of key-value stores and discusses strategies to minimize latency and ensure data integrity. We delve into the practical considerations for deploying such systems in real-world data centers, emphasizing the importance of maintaining robust performance under diverse network conditions. Our experimental results validate the effectiveness of our proposed approach, demonstrating substantial improvements in throughput and load reduction. Specifically, we highlight the efficacy of in-switch processing in optimizing write-

\*Corresponding author

intensive key-value store workloads.

This study addresses the critical challenge of cache invalidation caused by frequent writes in current key-value stores. By introducing a delayed-write mechanism, we offer an efficient and robust solution that significantly enhances the performance of in-network caches. Our approach mitigates the adverse effects of frequent data updates, ensuring the continued effectiveness of caching strategies. These insights underscore the potential of advanced in-switch processing techniques to meet the evolving demands of modern data centers. Our contributions can be summarized as follows:

- **Novel Delayed-Write Mechanism:** We introduce a delayed-write mechanism that effectively reduces the impact of cache invalidation and improves system throughput.
- **Lightweight Reliability and Consistency Mechanism:** We design a lightweight reliability mechanism that employs the heartbeat methods to recover lost data when encountering switch failure or state transitions.
- **Experimental Validation:** We provide detailed experimental results demonstrating the effectiveness of our approach in improving performance under write-intensive key-value stores workloads.

## II. BACKGROUND AND MOTIVATION

### A. Key-Value Stores and In-network Caching

Key-value stores are a cornerstone of modern data center infrastructure, providing a simple yet powerful mechanism for storing and retrieving data. They map unique keys to corresponding values, enabling efficient data access patterns for a variety of applications, including web services, content delivery networks, and distributed databases. To meet the demands of high-performance applications, key-value stores often employ caching strategies designed to reduce latency and increase throughput by keeping frequently accessed data in memory.

Traditional caching mechanisms, such as Least Recently Used (LRU), Most Frequently Used (MFU) and in-network caching, are optimized for read-heavy workloads. These strategies work by retaining popular data items in the cache, thus minimizing access times and reducing the load on backend storage systems. In environments where read operations dominate, these caching strategies can significantly enhance performance by ensuring that most read requests are served from the fast, in-memory cache.

### B. Challenges with Write-Intensive Workloads

However, the landscape of data workloads is changing. Modern applications increasingly involve write-intensive operations, where data updates occur frequently and at high volumes [3][6]. Examples include real-time analytics, logging systems, and social media platforms where user-generated content is continuously updated. In such environments, the benefits of read-optimized caching mechanisms are severely diminished.

Write-intensive workloads pose a fundamental challenge to traditional caching strategies: cache invalidation. When data is frequently updated, cached items quickly become outdated, necessitating constant invalidation and refresh operations. This high rate of cache turnover not only reduces the hit rate of the cache but also increases the overhead associated with maintaining cache coherence. As a result, the system is forced to fetch updated data from backend storage more often, negating the latency and throughput benefits that caching is supposed to provide.

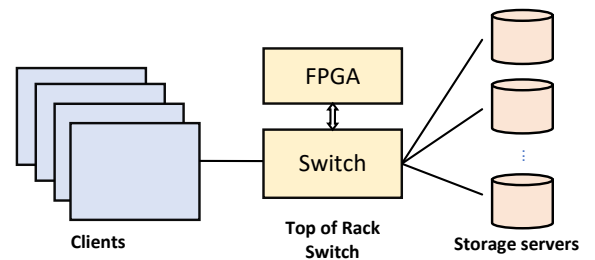
### C. Impact on System Performance

The impact of frequent cache invalidation on system performance can be profound. Increased backend storage accesses lead to higher latency and lower throughput. Moreover, the additional load on backend storage systems can create bottlenecks, further exacerbating performance issues. The overall efficiency of key-value stores is thus significantly compromised in write-heavy environments.

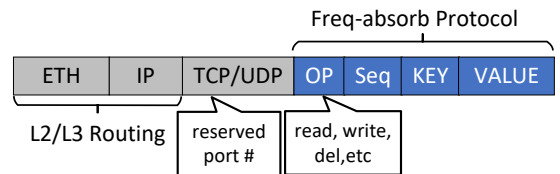
Centralized controllers, which are often used to maintain data consistency and synchronization within in-network caching systems, can become performance bottlenecks as well, as they have limited bandwidth to switches. This centralized approach limits the ability of the system to scale effectively, leading to further performance degradation as the volume of write traffic grows.

### D. Motivation for Freq-absorb

To address these challenges, there is a clear need for innovative solutions that can effectively handle write-intensive workloads without compromising the benefits of caching. The Freq-absorb architecture introduces a novel approach to mitigate the adverse effects of frequent writes on cache



(a) Freq-absorb architecture.



(b) Freq-absorb packet format.

Fig. 1: Freq-absorb overview.

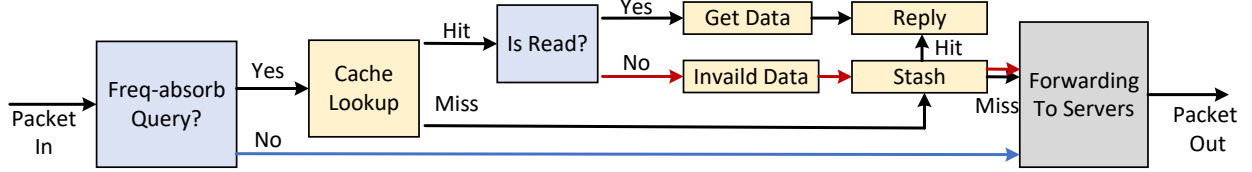


Fig. 2: The flow of key-value queries in the switch data plane.

performance. By leveraging in-switch processing capabilities, Freq-absorb implements a delayed-write mechanism that temporarily buffers write requests within the switch data plane.

This approach offers several advantages:

- **Reduced Cache Invalidation:** By delaying the commitment of write operations to the backend storage, the mechanism reduces the impact of cache invalidation on overall performance. This helps maintain higher switch hit rates and preserves the benefits of caching even under write-heavy conditions.
- **Pure Data Plane Processing:** Pure data plane processing alleviates the burden on centralized controllers. This approach allows for more efficient management of data versions, leading to improved overall performance.
- **Ensured Data Reliability and Consistency:** Employing a lightweight heartbeat mechanism, it can keep data reliability and consistency even when encountering switch failure or state transitions.

### III. SYSTEM DESIGN

In this section, we present Freq-absorb, a novel in-network key-value store to optimize the performance of the read cache under heavy writes. Freq-absorb aims to maintain the efficiency of in-network cache to handle write-intensive workloads.

#### A. Overview

The typical network architecture in data centers is the Spine-Leaf model, which separates computation and storage resources into different racks. Virtual machines or containers in the computation racks act as clients that read or write data on storage servers over the networks, typically using key-value stores. The architecture of Freq-absorb is consisted of clients, ToR switch, and storage servers, shown as in Fig. 1a. In the spine-leaf network, ToR switches are considered leaf nodes. Clients send key-value queries that must pass through the ToR switches to reach the backend servers. Therefore, most of our design is located in the ToR switch to optimize in-network caching when handling write-intensive workloads. We selected an FPGA-based switch as our ToR switch, which implements a read cache and an appended write buffer called stash.

We follow the network protocol design of prior work [5], which is based on TCP/UDP and appending OP, Seq, Key, and Values as payloads like Fig. 1b. Clients use either TCP or UDP based on their specific needs. There are 7

types of packets, where OP=4 and OP=5 packets support cache and write buffer functionalities, respectively, while the remaining correspond to different types of key-value stores queries. The three parties - clients, switches, and servers - will implement all mechanisms of Freq-absorb through interactions using these 7 types of packets, as shown in Table I. Freq-absorb is compatible with existing network protocols and infrastructure and can be easily integrated with existing key-value databases [7][8] with a slight overhead. It targets modern rack-scale storage clusters that contain thousands of cores per rack.

TABLE I: Queries Types

| OP | Query Type  | Referred HW  | Meaning          |
|----|-------------|--------------|------------------|
| 0  | READ        | Cache, Stash | Read query       |
| 1  | READ_REPLY  | -            | Read reply       |
| 2  | WRITE       | Stash        | Write query      |
| 3  | DELETE      | Cache, Stash | Delete query     |
| 4  | HOT_INSERT  | Cache        | Hot-items insert |
| 5  | STASH_SYNC  | Stash        | Stash Sync       |
| 6  | WRITE_REPLY | Stash        | Write reply      |

#### B. Delayed-write Mechanism

**Features of key-value stores workloads.** Before introducing delayed write, we need to understand the features of key-value stores workloads. Prior studies have shown that 10% of items account for 60 - 90% of queries in the Memcached deployment at Meta[4]. We refer to these 10% of items as "hot items". And read and write queries follow the Zipf distribution [9], the requested key-value pairs overlap significantly [5][6].

**Essence of Cache invalidation.** In-network caching involves setting up read caches within switches on the critical network path to handle read queries for hotspot key-values, thereby eliminating the need to forward these queries to the servers. This read cache periodically fetches hotspot key-value data from the servers. During read query processing, data is retrieved directly from the switch cache, enhancing overall system performance and reducing read latency. When the switch receives a write query, if the key-value is not present in the read cache, the query is forwarded directly to the server without any caching intervention. However, if the key-value exists in the read cache, the cache entry must be invalidated. This ensures that the server always holds the latest version of the data, maintaining both simplicity and data consistency in the system.

As the read/write ratio shifts in data centers, the proportion of write queries increases significantly. In-switch Read-

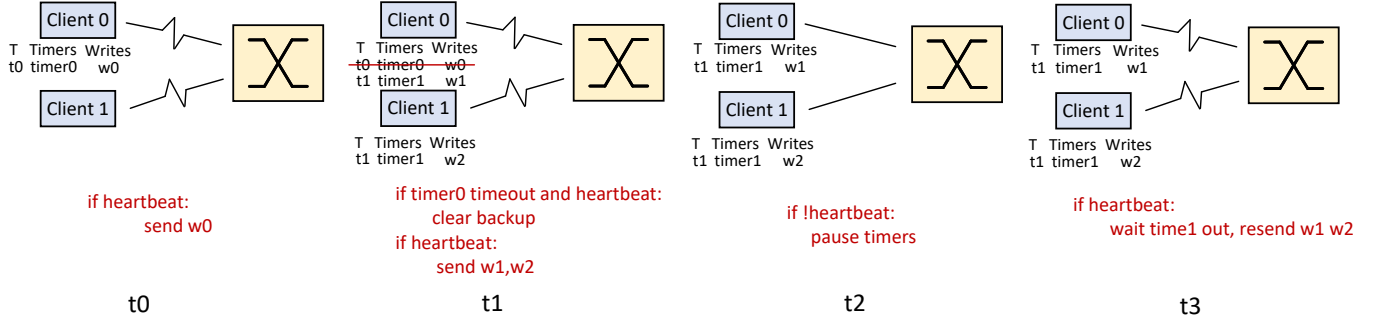


Fig. 3: The heartbeats mechanism. Note that  $t_0$ ,  $t_1$ ,  $t_2$ , and  $t_3$  exist in a temporal sequence, but they are not consecutive time intervals.

optimized caching experiences widespread hotspot item invalidation when the switch handles heavy writes and loses its benefit of serving hot read queries in the network. We observed that when a write query carrying the latest key-value data enters the switch, the initial action is to invalidate the corresponding entry in the read cache, followed by writing the data to the server. Subsequent read queries for the same key-value data, encountering a cache miss in the switch, are then forwarded to the server for retrieval.

We propose a new component, termed a stash (buffer), into the data plane of the switch to temporarily hold write queries, delaying their forwarding to the server side. The advantage of adding such a stash is that during the period when the write queries are temporarily stored in the stash, read queries for the invalidated key-value data in the cache can be fulfilled from the stash without the need to access the storage servers.

The process of handling key-value queries in the switch's data plane is illustrated in Fig. 2. Freq-absorb queries can be classified into three types: read, write, and delete. Since delete and write both involve updating key-values and share similar characteristics, this paper focuses only on read and write queries. When a packet enters the data plane, it is identified as a Freq-absorb query based on the reserved port field. If it is a Freq-absorb query, it is sent to the cache for lookup; otherwise, it is treated as irrelevant traffic and directly forwarded to the servers. For the cache lookup results: If a read query hits the cache, the requested data is retrieved from the cache and returned to the client. If it misses, the query is forwarded to the server. If a write query hits in the cache, the corresponding key-value entry in the cache is invalidated and the write query is temporarily stored in the stash. If it misses, the query is forwarded to the servers.

However, the duration for which write queries can be cached in the stash must be limited for two reasons. First, new writes continuously need to be stored in the stash, and its space is limited. Second, the memory in the switch is not non-volatile, so it cannot reliably and safely buffer write data for extended periods. We control the caching time by adjusting the size of the stash. When the stash is nearly full,

all writes are forwarded to the storage servers in the batched way. In our subsequent evaluation, we identified the optimal stash size for our experimental scenarios. We designed a queue data structure to serve as the stash. When the stash reaches an almost full state, all writes preceding the current write in the queue are sent to the servers.

### C. Data Consistency and System Reliability

In our delayed write mechanism design, the in-switch stash temporarily holds write requests that invalidate cache entries. At this point, the write requests contain the latest data, while the corresponding key-value data on the server is outdated and pending update. The server data becomes globally up-to-date only after the ToR switch forwards the write requests and completes the updates. However, in the event of a switch failure or state transition, the data in the in-switch stash can be permanently lost, leading to inconsistencies in the key-value store system and resulting in errors.

Based on the characteristics of in-network processing and key-value stores, we designed a reliability mechanism to ensure data consistency and system reliability. This mechanism ensures that data in the in-switch stash can be recovered in the event of a switch failure or state transition. Traditionally, after a client sends a write request, the system is expected to ensure the update is completed. However, in the delayed-write mechanism, when a write request resides in the in-switch stash, the client, after a slight modification, can recover the lost data in the event of a switch failure, thereby ensuring data consistency and system reliability.

We incorporated a heartbeat mechanism [10] commonly used in distributed systems into our design. The switch sends periodic heartbeat signals to all clients to indicate its operational status. If the heartbeat signals are interrupted, it indicates a switch failure. The detailed process of the reliability mechanism is illustrated in Figure 3. At time  $t_0$ , clients receive the switch's heartbeat signals, indicating no failure. Simultaneously, Client 0 performs three actions: sends write request  $w_0$ , backs up the  $w_0$  data, and sets a timer with a fixed interval  $P$ . By time  $t_1$ , if heartbeat signals have been continuously received between  $t_0$  and  $t_1$

and the w0 timer at Client 0 has timed out, the w0 backup is cleared, indicating that w0 has been successfully written to the servers. At t1, Client 0 and Client 1 send write requests w1 and w2, respectively, and set their respective timers. If the switch fails at t2, the clients will not receive the heartbeat signal. At this point, Client 0 and Client 1 pause their existing timers until the heartbeat signal resumes stability, and then continue timing.

At time t3, the switch's heartbeat signal resumes. Clients wait for the previously paused timers to time out, then send the backup write requests in the order of their timeouts and set new timers until the backup is cleared. Sending backups in strict timeout order ensures the sequence of multiple write requests, maintaining the consistency of updated data. Since multiple clients might send write requests simultaneously, the time taken to reach the ToR switch through the Spine-Leaf network architecture is considered equal. The precedence is determined by a predefined priority, which is not the focus of this paper.

In summary, our designed reliability mechanism uses the switch's heartbeat signals and client timeout retransmission to ensure that data lost in the in-switch stash can be recovered in the event of a switch failure or state transition. This approach guarantees data consistency and system reliability.

#### D. Formal Proof of Consistency

In this section, we provide formal proof of eventual consistency for the Freq-absorb architecture, which consists of clients, switches, and servers interacting under normal operation and failure conditions.

1) *Variables Definition*: The variables used in our proof are defined as follows:

TABLE II: Notation of Symbols

| Symbol          | Description                                                               |
|-----------------|---------------------------------------------------------------------------|
| $C$             | The set of clients.                                                       |
| $S$             | The set of servers.                                                       |
| $W_i(t)$        | Write request issued by client $C_i$ at time $t$ .                        |
| $T_{client}$    | The client timer timeout period.                                          |
| $R_{write}$     | The write request sending rate.                                           |
| $\alpha$        | The read-write ratio (proportion of write requests to total requests).    |
| $T_{stash}$     | The switch buffering time.                                                |
| $f$             | The heartbeat frequency.                                                  |
| $P_{failure}$   | The probability of switch failure.                                        |
| $T_{propagate}$ | The time for a write request to propagate from the client to all servers. |
| $T_{switch_r}$  | The switch receive time.                                                  |
| $T_{server_r}$  | The server receive time.                                                  |
| $T_{retrans}$   | The retransmission time.                                                  |

2) *Write Request Propagation Model*: The client  $C_i$  issues a write request  $W_i(t_{issue})$  at time  $t_{issue}$  and retains a backup, setting a timer with a timeout period  $T_{client}$ . The switch receives  $W_i(t_{issue})$  at time  $T_{switch_r}$  and commits it

to the server after a buffering time  $T_{stash}$ , so the commit time is:

$$T_{commit} = T_{switch_r} + T_{stash} \quad (1)$$

The switch sends heartbeat signals to the client at frequency  $f$ , and if no switch failure or state transition, the server receives  $W_i(t_{issue})$  at time

$$T_{server_r} = T_{commit} + T_{switch \rightarrow server} \quad (2)$$

and updates its storage. To guarantee that every write request  $W_i(t_{issue})$  will eventually be committed to the server, we define the final commit time as

$$t_{final\_commit} = t_{issue} + T_{propagate} \quad (3)$$

where

$$T_{propagate} = T_{server_r} + n \times T_{client} \quad (4)$$

The number of retries  $n$  is given by

$$n = \left\lceil \frac{T_{propagate} - T_{server_r}}{T_{client}} \right\rceil \quad (5)$$

When the switch fails, it takes  $T_{recover}$  time to recover. During this period, clients pause their timers. Once the switch recovers, clients restart their timers and retransmit lost write requests sequentially after  $T_{client}$  timeout, leading to

$$T_{retrans} = t_{issue} + T_{recover} + n \times T_{client} \quad (6)$$

3) *Consistency in fault-tolerant scenario*: Each write request  $W_i(t_{issue})$  will eventually be committed to the servers at time  $t_{final\_commit}$ . Therefore, we have

$$\forall W_i(t), \exists t' \geq t + T_{propagate}, \forall S_j \in S, W_i(t) \in S_j \quad (7)$$

The probability of switch failure is  $P_{failure}$ , and the heartbeat frequency is  $f$ . The client detects failure through heartbeat signals and retransmits the write request, with failure detection time  $1/f$ . The retransmission time is

$$T_{retrans} = t + T_{recover} + n \times T_{client} \quad (8)$$

During the failure detection period, the clients continue to issue write requests. The number of write requests issued during the failure detection time  $1/f$  is

$$R_{write} \times \alpha \times \frac{1}{f} \quad (9)$$

These write requests are lost due to the switch failure. Once the switch recovers, the clients restart their timers and retransmit these lost write requests after  $T_{client}$  timeout. The propagation time  $T_{propagate}$  includes the time for these retransmissions, ensuring that all write requests are eventually committed. Therefore, any read request  $R_i(t_{read})$  issued at  $t' \geq t + T_{propagate}$  will return the latest written value, i.e.,

$$\forall R_i(t'), t' \geq t + T_{propagate}, R_i(t') = W_{latest}(t) \quad (10)$$

4) *Proof of Eventual Consistency*: Each write request  $W_i(t_{issue})$  will eventually be committed to the server at time  $t_{final\_commit}$ . Even if the switch fails, the client will retransmit the write request after  $T_{client}$  timeout until it is successfully committed. The propagation time  $T_{propagate}$  ensures that all write requests are eventually visible on all servers, ensuring data consistency. The heartbeat mechanism ensures that clients can promptly detect switch failures and retransmit write requests, preventing data loss. Any read request issued after  $t_{issue} + T_{propagate}$  will return the latest written value, ensuring that the system achieves eventual consistency.

By analyzing the above mathematical relationships and equations, we can formally prove that the Freq-absorb architecture ensures eventual consistency even encountering switch failure.

#### IV. DISCUSSION

##### A. Is the client overhead affordable?

From a holistic system perspective, clients bear the task of setting up backups and timeout retransmission, which adds additional overhead. However, from the perspective of a single client, it only bears the overhead related to its own sent write requests, which depends on the number of write requests sent. Especially in typical distributed systems like key-value stores, where there are numerous clients, the overall overhead is shared among multiple clients. Therefore, the overhead of the new mechanism on the client side is manageable.

##### B. The delay for introducing stash.

Introducing a stash to temporarily buffer some write requests objectively increases the latency of key-value data writing to the server. However, if we consider the ToR switch as part of the storage servers, functioning similarly to a server cache layer, the difference between temporarily storing data in the ToR switch and subsequently updating it to the server through a reliability mechanism versus directly sending it to the server for persistent storage becomes negligible from the client's perspective.

Moreover, according to our calculations, when the stash size is 1000 items, the delay introduced by the stash is at the microsecond level. Compared to the significant improvement in switch miss rates and the overall performance boost, the additional delay is acceptable.

#### V. EVALUATION

In this section, we evaluate Freq-absorb and analyze the switch miss and overall performance under different conditions, including with and without stash, different workload skewness, and varying read/write ratios. The results demonstrate that Freq-absorb provides significant performance improvements on throughput for high-performance key-value stores, and effectively decreases switch miss rates under write-intensive workloads.

##### A. Experimental setup

**Testbed.** Our testbed consists of a NetFPGA-SUME, a switch, and two physical machines. One machine is used as a client to generate key-value queries, while another as the storage server.

**Workloads.** We adopt an approximate approach to synthesize workloads that conform to the Zipfian distribution, characterized by varying degrees of skewness (0.99, 0.95, and 0.90). Observations from real-world operational deployments confirm an alignment with the Zipfian distribution. Consequently, our experimental design predominantly incorporates both skewed and uniform workloads to evaluate the efficacy of designed key-value stores.

Keys are randomly chosen within the scope of 1 to 100,000 and are consistently hashed to divide the key-space into partitions. Each key has a uniform length of 16 bytes, and the value is standardized to 128 bytes, which mirrors the typical size found in Twitter's clusters. The experiments are conducted using a stash of varying sizes and a cache that can accommodate 1,000 entries.

**Machine Settings.** The switch is equipped with 32 10Gbps ports. It connects the client and server via optical fiber in a snake way described in [5]. Instead of routing key-value traffic directly to the servers, it directs such traffic to the FPGA for processing. To assess the system's performance enhancement attributed to the stash and other experimental goals, multiple servers are required. Given the constraint of a limited number of available machines, we have configured 32 queues on a single physical server to emulate 16 client and 16 storage server entities, each responsible for handling a segment of the key-value queries. The client's sending rate of the clients is calibrated to align with this arrangement to mitigate packet loss within the queues. However, since all queues are linked to a single physical NIC on the same machine, the throughput of the simulated server is reduced to 1/16 of the original throughput. To compensate for this reduction and derive a more accurate measure of performance, the results are scaled up by a factor of 16. This setup is reasonable because we are more interested in the relative changes in performance with respect to metrics rather than absolute performance numbers.

##### B. Switch miss vs. Write ratio

Switch Miss represents the ratio of client queries not satisfied by the switch to the total number of requests. A lower value indicates more effective in-network caching. We conducted tests under four different read-write ratios: 100% read, 10% write (90% read), 30% write, and 50% write. For each group, we tested the Switch Miss ratio under four skewed load distributions: uniform, Zipf-0.90, Zipf-0.95, and Zipf-0.99. The experimental results are shown in Fig. 4.

In Fig. 4a, where client queries are 100% read, setting up in-network cache effectively reduces Switch Miss across four different skewed loads. When the load skew is highest (Zipf-0.99), Switch Miss drops to 55.1%. Starting from Fig.



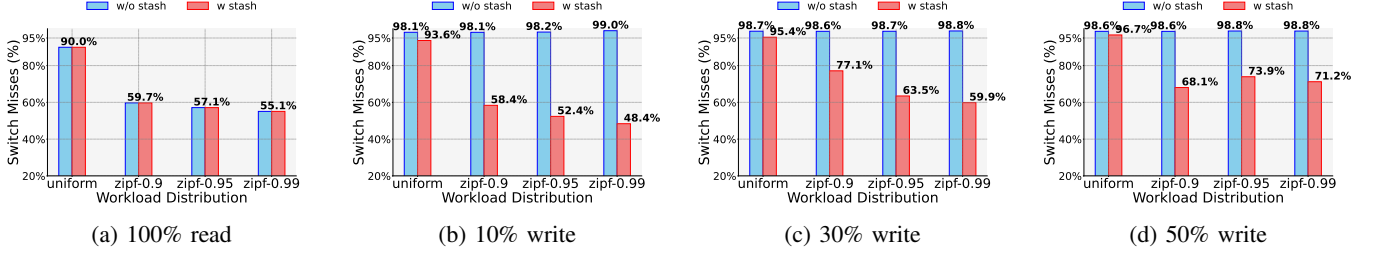


Fig. 4: Switch Miss vs. Write ratio

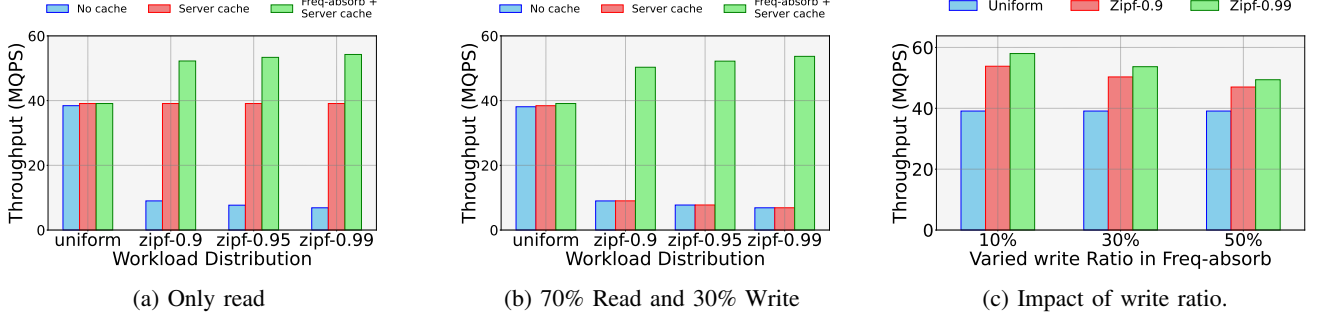


Fig. 5: Throughput vs. skewness and write ratio

4b, the in-network cache needs to handle write requests, and the cache benefits almost disappear; without the stash, Switch Miss exceeds 98% across all four load distributions. However, with our designed stash, Switch Miss drops significantly when handling skewed loads, reaching 48.4% at the highest load skew (Zipf-0.99). Fig. 4c and 4d show similar conclusions, but as the write ratio increases, the gains from the stash diminish. This is likely because the proportion of read queries decreases, leading to more unmet read queries and write requests being forwarded to the backend servers.

### C. System throughput

**Skewness.** Fig. 5a illustrates system throughput under skewness parameters of 0.9, 0.95, and 0.99 with 100% read queries and 1K hot items in the cache. We compare Freq-absorb with NoCache and server cache under different workloads. The server cache maintains steady performance despite skewed workloads, while NoCache performs poorly, with throughput dropping to 19.99% (0.95) and 17.89% (0.99) of that under uniform workloads.

For workloads with 70% read and 30% write (write-intensive), write queries follow the same distribution as read queries. Fig. 5b shows that server cache throughput drops to 23.47% under skewed loads due to write queries invalidating cached items, causing cache misses for read queries. NoCache performance degrades similarly. Freq-absorb, however, uses a stash module to buffer writes and delay execution. Missed read queries check the stash before hitting the server, allowing Freq-absorb to reach 58 MQPS under Zipf-0.99, a 1.37x improvement over the baseline.

**Write Ratio.** We tested the overall system performance under different read-write ratios when handling loads with varying degrees of skewness. Fig. 5c shows that the performance is highest under a load with 10% writes, and performance decreases as the proportion of write requests in the load increases. However, performance under skewed loads remains better than that under uniform loads. This trend is consistent with Fig. 4, where Switch Miss increases as the write ratio increases.

## VI. RELATED WORK

This section reviews the key areas of related work, focusing on in-memory key-value stores, and recent advances in in-network processing.

### A. In-Memory Key-Value Stores

In-memory key-value stores like Redis and Aerospike have been developed to leverage the high-speed access provided by RAM. These systems achieve low-latency data access by keeping the entire dataset, or at least a significant portion of it, in memory. Redis, for example, supports complex data structures and provides mechanisms for persistence and replication, which help maintain data integrity.

### B. In-network Processing

Recent advancements in network technology have introduced the concept of in-network processing, where network switches are equipped with programmable capabilities. Projects like P4 [11] (Programming Protocol-Independent Packet Processors) have enabled the development of switches that can perform custom processing on

packet flows. NetCache [5] is an example of an in-network key-value store that leverages programmable switches to cache frequently accessed items within the network. This approach reduces latency by bringing data closer to the client. However, NetCache and similar systems primarily focus on read optimization and do not fully address the challenges posed by write-intensive workloads.

FarReach [4] recognizes the growing prevalence of write-oriented queries and addresses the challenge of write-intensive workloads by introducing a write-back caching mechanism. This mechanism is complemented by a co-designed controller and data plane. However, the deployment faces a significant performance obstacle due to the constrained bandwidth between switches and the controller, which is critical as the controller may have to interact with multiple switches concurrently.

### C. FPGA Integration

The use of FPGAs for accelerating data-intensive operations has gained traction in recent years. FPGAs offer the flexibility to implement custom processing pipelines that can handle complex tasks with low latency. Catapult [12], a project by Microsoft, integrated FPGAs into their data centers to accelerate search and machine learning workloads, demonstrating significant performance improvements. In the context of key-value stores, FPGAs have been used to offload specific tasks such as compression and encryption. For instance, KV-Direct [13] leverages FPGAs to accelerate key-value operations by offloading them to programmable NICs. However, it does not leverage the highly skewed feature of workloads. If combined with the aforementioned in-network caching methods, it could achieve higher system performance.

The Freq-absorb architecture builds on these hybrid approaches by integrating in-switch processing with FPGA acceleration to specifically address the challenges of write-intensive workloads. By delaying writes within the switch data plane, Freq-absorb minimizes the impact of cache invalidation and maintains high switch hit rates, providing a scalable and efficient solution for modern data centers.

## VII. CONCLUSIONS

This paper introduces Freq-absorb, a novel key-value store architecture designed to optimize the efficiency of in-network caching for key-value stores under write-intensive workloads, where traditional caching strategies fail. By employing a delayed-write mechanism that buffers writes within FPGA-augmented switches, Freq-absorb minimizes the impact of cache invalidation and maintains high switch hit rates. This decentralized approach not only reduces the load on backend storage but also alleviates bottlenecks associated with centralized control mechanisms. Our experimental results confirm that Freq-absorb significantly improves system throughput and reduces the load sent to servers when handling write-intensive workloads, making it a robust and

efficient solution for modern data centers. This work paves the way for further exploration of in-network processing in optimizing data-intensive applications.

### ACKNOWLEDGMENT

We thank the anonymous reviewers for their constructive feedback and suggestions for improving this work. This work is supported by the National Science Foundation of China under grant NO.62372462.

### REFERENCES

- [1] F. Chang, J. Dean, S. Ghemawat, *et al.*, “Bigtable: A distributed storage system for structured data,” *ACM Transactions on Computer Systems*, vol. 26, no. 2, pp. 1–26, 2008.
- [2] S. Idreos and M. Callaghan, “Key-value storage engines,” in *Proceedings of the 2020 ACM SIGMOD*, 2020, pp. 2667–2672.
- [3] J. Yang, Y. Yue, and K. V. Rashmi, “A large-scale analysis of hundreds of in-memory key-value cache clusters at twitter,” *ACM Trans. Storage*, vol. 17, no. 3, Aug. 2021.
- [4] S. Sheng, H. Puyang, Q. Huang, L. Tang, and P. P. Lee, “{Farreach}: Write-back caching in programmable switches,” in *USENIX ATC 23*, 2023, pp. 571–584.
- [5] X. Jin, X. Li, H. Zhang, *et al.*, “Netcache: Balancing key-value stores with fast in-network caching,” in *Proceedings of the 26th SOSP*, 2017, pp. 121–136.
- [6] Z. Cao, S. Dong, S. Vemuri, and D. H. Du, “Characterizing, modeling, and benchmarking {rocksdb}{key-value} workloads at facebook,” in *18th USENIX FAST 20*, 2020, pp. 209–223.
- [7] LevelDB, <https://github.com/facebook/rocksdb>.
- [8] RocksDB, <https://github.com/facebook/rocksdb>.
- [9] J. Gray, P. Sundaresan, S. Englert, K. Baclawski, and P. J. Weinberger, “Quickly generating billion-record synthetic databases,” in *Proceedings of the 1994 ACM SIGMOD*, 1994, pp. 243–252.
- [10] F.-f. Li, X.-z. Yu, and G. Wu, “Design and implementation of high availability distributed system based on multi-level heartbeat protocol,” in *2009 IITA*, 2009, pp. 83–87.
- [11] P. Bosshart, D. Daly, G. Gibb, *et al.*, “P4: Programming protocol-independent packet processors,” *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.
- [12] A. Putnam and e. a. Caulfield Adrian M., “A reconfigurable fabric for accelerating large-scale datacenter services,” ser. ISCA ’14, 2014, pp. 13–24.
- [13] B. Li, Z. Ruan, W. Xiao, *et al.*, “Kv-direct: High-performance in-memory key-value store with programmable nic,” in *Proceedings of the 26th SOSP*, 2017, pp. 137–152.